Smart waste sorting and bin monitoring

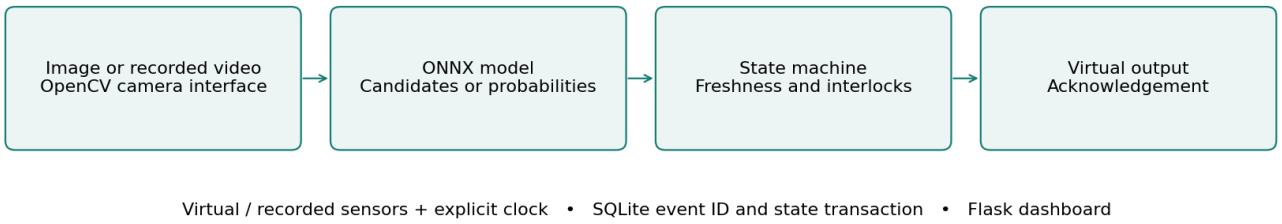
Technical report and reproducible evaluation

11 November 2026 | Abhijith Viswanathan | AI-assisted implementation

A frozen MobileNetV3-small feature extractor and a trained linear classifier recognize cardboard, glass, metal, paper and plastic. Confidence rejection, mixed-material flags and bin checks can route an item to review. A disposal is counted only after actuator acknowledgement; jams, stale sensors and timeouts stop the simulated mechanism.

Raspberry Pi targeted, desktop simulation tested. No Pi, real sensor, electrical power, camera timing, motor or physical safety validation was performed. This is an AI-assisted personal engineering project; it does not establish professional seniority or independent authorship.

Architecture



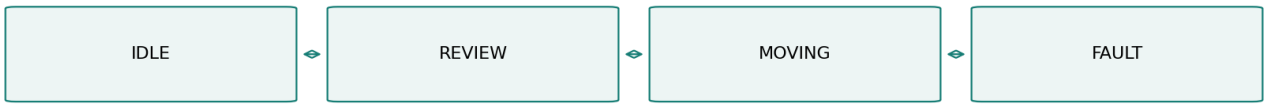
The application keeps perception, control and hardware interfaces separate. Virtual inputs and the explicit clock make decisions replayable; SQLite commits event identifiers and controller state together. The dashboard runs locally and never opens a webcam unless explicitly enabled.

Main measured comparison

Known material accuracy: 62.168 to 90.265 percent. 452 held-out known images; HSV linear baseline vs frozen MobileNet features + trained linear head.

The following pages separate real image evaluation, host execution time, scripted control outcomes and future physical validation. Raw observations, exact selections, source attribution and regression logs accompany this report.

Control behavior and recovery



State inventory; transition guards and recovery paths are defined in the policy table and core.py

1. IDLE accepts a new item only with valid five-class probabilities, configured confidence and margin, and healthy capacity sensors.

2. Unknown, mixed or full-bin inputs enter REVIEW. Fill >=90% or weight >=2000 g inhibits routing.

3. MOVING counts exactly once on acknowledgement. Virtual acknowledgement occurs after one simulated second; manual acknowledgement is also available.

4. Jam, invalid sensors, capacity becoming full, or a three-second timeout enters FAULT before acknowledgement. Reset returns to REVIEW. Restart never automatically resumes motion.

Freshness and persistence

For classroom and parking, real image evidence expires after two simulation seconds at a new check; advancing time cannot make a stored frame fresh. Restart invalidates camera evidence. A gate interrupted during a cycle enters FAULT, and an interrupted waste move requires review. Manual simulation inputs are explicitly labeled fixtures.

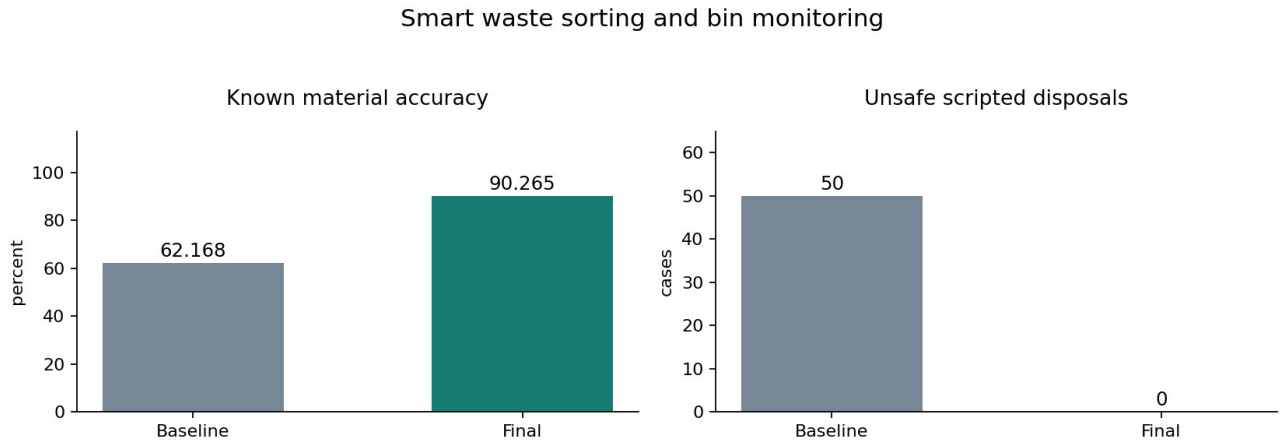
The classroom and parking tracker uses class-aware one-to-one overlap association, a five-second expiry, two-hit confirmation and a line deadband. An 80-frame synthetic trajectory produced one unique ID and one expected entry. This verifies mechanics, not tracking accuracy on crowded video. IDs and entry counters persist, but active tracks do not survive restart.

A state change and its event ID are stored in one SQLite transaction. Duplicate event IDs are ignored. Failed persistence rolls back in-memory controller state and newly accepted inference/tracker state. This cannot undo a real actuator movement; the physical design requires independent acknowledgement and reconciliation.

State-machine source: docs/state-machine.mmd. Controllers: lab/core.py. Hardware interfaces: lab/hal.py. The current app uses serialized, synchronous inference and virtual-clock control; it is not a real-time motor controller.

Models data and measured tradeoffs

Training used 1457 known-class images, 478 known validation images and 452 known test images. Unknown-proxy trash counts are 39 validation and 21 test; 77 train-bucket trash images were unused. SHA256 groups exact duplicates into fixed roughly 60/20/20 buckets. A 576-feature frozen ImageNet MobileNetV3-small backbone feeds StandardScaler and balanced L2 multinomial logistic regression. Only the scaler and linear head were fitted on training data. C in {0.01, 0.1, 1} and rejection settings were selected on validation. Frozen C=0.01, threshold=0.5, margin=0.0. No CNN fine-tuning or training from scratch occurred.



Known-class accuracy rose from 62.17% to 90.27%, a 28.10 percentage-point gain (45.20% relative). The feature extractor costs more computation than a color histogram. The selected rejection rule recognized only 4 of 21 unknown-proxy images, or 19.05%. A confident prediction cannot establish purity or suitability for automatic disposal.

TrashNet contains mostly isolated objects and is not representative of dirty mixed waste streams. Exact duplicates are grouped, but physical-object/session identifiers are unavailable, so near-duplicate leakage cannot be excluded. Trash is a narrow proxy for unfamiliar items. In 60 mixed-image composites, only 16 were rejected. Dirt, resampling and black masks are synthetic stress probes, not physical crushing, dirt or hidden contamination measurements.

Measurement protocol and verification

Metric	Baseline	Final	Unit
Known material accuracy	62.168	90.265	percent
Known material macro F1	0.617	0.901	score
Unsafe scripted disposals	50.000	0.000	cases

Known material accuracy: 452 held-out known images; HSV linear baseline vs frozen MobileNet features + trained linear head.

Known material macro F1: Same 452 images; five material classes; forced-choice before rejection.

Unsafe scripted disposals: 60 scripted score/fault cases; 10 normal acknowledgements.

Complete local inference requests: median 5.35 ms and p95 5.72 ms over 30 repeated requests after one cold request (11.62 ms). Flask test-client timing includes image decode, inference, annotation, controller update, SQLite and JSON; it excludes network and browser rendering. The same selected image was repeated. CPU and cumulative process memory samples are in evidence/request-timing.json.

Environment: macOS 26.2 arm64, Python 3.12.14, ONNX Runtime 1.21.0 CPU, OpenCV 4.11.0.86 and NumPy 2.2.6. Detector timing separately compares 36 calls at each size; median 13.31 ms at 320 and 48.50 ms at 640. Short host runs do not predict Pi throughput, thermal behavior or sustained response bounds.

Regression result: 56 shared cases passed, covering state guards, invalid and stale input, manual requests, duplicate IDs, restart, large time steps, failed commits and HTTP input boundaries. The same suite is included in each standalone project; these are not 168 distinct tests. Browser checks exercised real inference and the advertised failure controls.

Version history preserves the initial 38 passing core cases, subsequent API and review regressions, and the final 56-case run. Decoder correction, stale image handling, obstruction-source separation, capacity changing during motion, scenario labeling and restart/rollback fixes are recorded in docs/FAILURES.md. Earlier detection results remain available and are not used as final metrics.

Hardware integration and physical validation

Raspberry Pi 4 or 5, USB camera and fixed illumination, small bench servo, distance sensor for fill, load cell with a calibrated USB serial bridge, position/limit switch and jam input, separate regulated servo supply.

Example BCM allocation: fill trigger GPIO23 and echo GPIO24 through suitable level conversion; servo signal GPIO18; jam input GPIO17; position acknowledgement GPIO27. Weight readings arrive on a USB serial bridge at 115200 baud as one calibrated gram value per line. Direct HX711 support is not implemented.

Interface	Implemented now	Required before physical use
Camera	Recorded files and explicit USB camera option	Capture cadence and disconnect validation
Sensor readings	Virtual and recorded value valid at records	Calibrated GPIO or serial acquisition
Outputs	VirtualOutput and optional GPIO/servo classes	State-to-pin mapping and physical acknowledgement
Timing	VirtualClock and MonotonicClock interfaces	Separate control scheduling and deadline measurements
Persistence	SQLite event and state transaction	Power-loss and incomplete-actuation reconciliation

Required bench tools are a digital multimeter, logic analyzer and USB power meter; an oscilloscope is useful for supply transients. Use BCM numbering, verify 3.3 V GPIO levels and level-shift 5 V echoes. A servo needs a suitable separate supply and common signal ground. The lighting demonstration uses a low-voltage LED, and the gate is an unloaded bench mechanism.

The shipped app does not activate GPIO. Optional adapter classes are integration building blocks. A target runner must timestamp real acquisitions, disable synthetic acknowledgements, map outputs, and keep interlock checks independent of inference. Direct HX711 support is not included; weight expects a calibrated serial bridge.

Validate in stages: inspect disconnected rails; calibrate sensors; inject wiring and sensor faults; measure LED/servo command and stop timing; run sustained capture under load; interrupt power during writes and movement; then collect deployment data split by complete recording and location. Every physical test remains pending.

Hardware sources: Raspberry Pi computer documentation and gpiozero input/output API documentation. Full URLs, pin notes and the integration checklist are in docs/hardware.md.

Dashboard demonstration and learning

Panel	Interpretation
Input and detections	Uploaded image/recorded frame with boxes, or a clearly labeled synthetic scene.
Observation	Controller state, distinct from detector confidence.
Count and actuator	Candidate count/free slots/acknowledged disposals; virtual output only.
Last inference	Measured model/preprocessing time on this host; not Pi latency.
Latest decision and sensors	Reason, validity and current virtual sensor values.
Observation history	Count versus explicit simulation time; duplicate timestamps may overlap.
Experiment controls	Fault scenarios, manual requests, threshold and applicable interval controls.
Event ledger	Latest 30 rendered events from a 500-event export window; database retains all events.
Recorded benchmarks	Frozen historical results and JSON export; not recomputed by dashboard controls.

Choose paper and Run AI on sample. Sort latest AI result then advance one second to count acknowledgement. Click unknown to route scripted uncertainty to REVIEW; full to inhibit sorting; jam to enter FAULT. Restore devices and Clear fault to return to review; a new item requires a new decision.

Start with Python 3.12, create .venv, install requirements.txt and run app.py. Control fixtures need no model files. For AI, install requirements-training.txt and run tools/setup_models.py and tools/fetch_samples.py. The trained waste head is included; source packages omit pretrained backbones and COCO photos. Detailed commands and troubleshooting are in docs/setup-and-demo.md.

Use docs/mentor-guide.md for the code walkthrough, engineering alternatives, ten explained interview questions, modifications and three measured resume bullets. Begin with core.py and its tests, then trace a dashboard action through runtime.py, inference.py and the persistence interface.

Attribution: Ultralytics YOLO11, COCO, TrashNet by Gary Thung and Mindy Yang, and torchvision MobileNetV3. License texts and source links are retained in NOTICE.md, licenses and docs/model-and-data.md. No identity recognition is implemented. The project demonstrates reproducible simulation work; physical deployment and senior professional experience are not claimed.